



به نام خدا



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

هوش مصنوعی قابل اعتماد

تمرین اول

نام و نام خانوادگی	سید محمد جواد شریفی
شماره دانشجویی	۸۱۰۱۰۲۴۶۹
تاریخ ارسال گزارش	۱۴۰۵/۲/۱۵

4	سؤال اول: تعمیم پذیری
4	بخش اول: سوالات توضیحی
4	زیربخش اول: مفهوم همبستگی کاذب و شکست ERM
4	زیربخش دوم: فرموله سازی خطای بدترین گروه و گروه های داده
5	زیربخش سوم: نقش تابع زیان GCE
6	زیربخش چهارم: بررسی فرض اول مقاله
6	زیربخش پنجم: ایده الگوریتم JTT
8	بخش دوم: سوالات پیاده سازی
8	زیربخش اول: آماده سازی داده و مصور سازی
8	زیربخش دوم: آموزش مدل پایه با ERM
9	زیربخش سوم: استخراج مجموعه خطا (ERROR SET)
9	زیربخش چهارم: ساخت دیتاست وزن دار (UPSAMPLING)
9	زیربخش پنجم: آموزش مجدد مدل
10	زیربخش ششم: پیاده سازی تابع زیان GCE
11	سوال دوم: حملات خصمانه
11	بخش اول: مرور مفاهیم
11	سوال اول
11	سوال دوم
11	سوال سوم
12	بخش دوم: حمله جعبه سیاه
12	سوال اول
12	سوال دوم

12.....	سوال سوم.....
13.....	سوال چهارم.....
13.....	سوال پنجم.....
14	بخش سوم: پیاده‌سازی حملات جعبه‌سفید.....
14.....	زیربخش اول: مدل و مجموعه داده.....
15.....	زیربخش دوم: FGSM.....
17.....	زیربخش سوم: PGD.....
19	مراجع.....

سؤال اول: تعمیم پذیری

بخش اول: سوالات توضیحی

زیربخش اول: مفهوم همبستگی کاذب و شکست ERM

همبستگی کاذب به رابطه‌ای گمراه‌کننده گفته می‌شود که بین برچسب اصلی و یک ویژگی فرعی در داده‌های آموزشی زیاد دیده می‌شود؛ اما این ویژگی، واقعا یک ویژگی اصلی کلاس نیست.

مثلا در تشخیص گاو و شتر، پس‌زمینه‌ی بیابان ممکن است به اشتباه یک ویژگی اصلی از کلاس شتر در نظر گرفته شود. در حالیکه چنین نیست و صرفا یک همبستگی کاذب بوده است.

اساس روش ERM این است که میانگین خطا روی کل داده‌ها را کم می‌کند. بنابراین این روش تحت تاثیر اکثریت داده‌ها قرار می‌گیرد. پس اگر بیشتر داده‌ها bias-aligned باشند، یعنی دارای همبستگی‌های کاذب باشند، مدل یاد می‌گیرد که از همین همبستگی‌ها استفاده کند و روی این ویژگی‌ها train می‌شود؛ چرا که این کار برای کاهش loss داده‌های آموزشی ساده‌تر است. بنابراین مدل به جای ویژگی‌های اصلی، روی ویژگی‌های کاذب overfit می‌شود و در نمونه‌هایی که همبستگی کاذب در آنها برقرار نیست و به اصطلاح مقاله bias-conflicting هستند، دچار اشتباه می‌شود.

زیربخش دوم: فرموله‌سازی خطای بدترین گروه و گروه‌های داده

فرمول min-max ارائه شده در این مقاله به صورت زیر است:

$$\min_{\theta} \max_{b \in \mathcal{B}} \left\{ \hat{\mathcal{L}}_b := \frac{1}{n_b} \sum_{(x,y,b) \in G_b} \ell(f_{\theta}(x), y) \right\}$$

شکل ۱: فرمول min-max

با توجه به مقاله، نمونه‌های b_a نمونه‌های هم‌راستا با بایاس هستند. به این معنا که همبستگی کاذب در این نمونه‌ها برقرار است. در واقع این نمونه‌ها شامل همان ویژگی‌های فرعی هستند که همبستگی زیادی با برچسب اصلی آنها دارند و می‌توانند مدل را دچار خطا کنند.

در مقابل اما نمونه‌های b_c نمونه‌های در تضاد با بایاس نام دارند. به این معنا که همبستگی کاذب در آنها وجود ندارد و فاقد ویژگی‌های مذکور هستند.

زیربخش سوم: نقش تابع زیان GCE

علت استفاده از Generalized Cross-Entropy (GCE) به جای Cross-Entropy (CE) این است که در صورت استفاده از GCE، تأثیر گرادیان نمونه‌هایی که احتمال پیش‌بینی درست آنها بالا است، در آموزش مدل بیشتر می‌شود. این نمونه‌ها (نمونه‌هایی که احتمال پیش‌بینی درست آنها بالا) در واقع همان نمونه‌های هم‌راستا با بایاس هستند که دارای همبستگی کاذب می‌باشند و مدل دقت خیلی خوبی بر روی آنها دارد. بنابراین مدلی که با GCE آموزش داده می‌شود، عمداً به سمت یادگیری ویژگی‌های بایاس‌دار و همبستگی‌های کاذب سوق داده می‌شود و این باعث می‌شود که در ادامه تفکیک بهتری بین نمونه‌های b_c و b_a صورت بگیرد و در نتیجه با دادن وزن بیشتر به نمونه‌های b_c و آموزش دوباره مدل، به هدف نهایی مقاله برسیم.

رابطه تابع زیان GCE به صورت زیر است:

$$\ell_{GCE}(f_{\phi}(x), y) = \frac{1 - p_{\phi}(y|x)^q}{q}$$

شکل ۲: تابع زیان GCE

از همین رابطه می‌توانیم متوجه شویم که چرا تأثیر گرادیان نمونه‌هایی که احتمال پیش‌بینی درست آنها بالا است، در GCE نسبت به CE بیشتر است:

$$\frac{\partial \ell_{CE}}{\partial p} = -\frac{1}{p}$$

$$\frac{\partial \ell_{GCE}}{\partial p} = -p^{q-1}$$

بنابراین رابطه زیر را بین گرادیان‌ها داریم:

$$\nabla \ell_{GCE} = p^q \cdot \nabla \ell_{CE}$$

چون p همان احتمال مدل برای برای برچسب درست است، برای نمونه‌هایی که احتمال پیش‌بینی درست آنها زیاد است، مقدار p^q مقدار نسبتاً بزرگی می‌شود و بنابراین می‌توان نتیجه گرفت که تأثیر این نمونه‌ها در train کردن مدل، در حالت استفاده از GCE نسبت به CE بیشتر است و به همین دلیل برای این مقاله از GCE استفاده شده است.

زیربخش چهارم: بررسی فرض اول مقاله

فرض اول مقاله می‌گوید که میانگین خطای مدل روی نمونه‌های هم‌راستا با بایاس کمتر از میانگین خطای آن روی نمونه‌های در تضاد با بایاس است. یعنی:

$$\hat{\mathcal{L}}_{b_a} < \hat{\mathcal{L}}_{b_c}$$

شکل ۳: مقایسه میانگین خطای انواع نمونه‌ها

مفهوم این فرض این است که مدل، (مدلی که به همبستگی‌های کاذب وابسته شده) بر روی نمونه‌های دارای همبستگی کاذب، نسبت به نمونه‌های فاقد این نوع همبستگی بهتر عمل می‌کند.

اهمیت این فرض در بازنویسی تابع هدف، این است که اگر این فرض را بپذیریم، تابع min-max هدف به یک تابع زیان وزن‌دار تبدیل می‌شود که بررسی آن را ساده‌تر می‌کند. در واقع در تابع هدف min-max، به دنبال کمینه کردن بیشترین loss در بین همه گروه‌ها هستیم؛ حال اگر فرض اول را بپذیریم، گروهی که بیشترین loss را دارد گروه b_c است و بنابراین به جای آنکه min-max را روی همه گروه‌ها انجام دهیم به دنبال کمینه کردن یک تابع زیان وزن‌دار هستیم که در آن وزن هر نمونه متناسب است با احتمال تعلق آن نمونه به b_c :

$$\min_{\theta} \sum_{(x,y,b) \in \mathcal{D}} r(x,y,b) \ell(f_{\theta}(x), y).$$

شکل ۴: تابع هدف بازنویسی شده

همچنین این فرض در تشخیص نمونه‌های سخت که در واقع همان نمونه‌های در تضاد با بایاس هستند، ضروری است. چرا که در این مقاله، تشخیص نمونه‌های سخت با استفاده از اختلاف بین برچسب واقعی و پیش‌بینی مدل انجام می‌شود. در واقع هر چه احتمال پیش‌بینی درست مدل برای یک نمونه کمتر باشد، احتمال تعلق آن نمونه به b_c بیشتر می‌شود. پس در واقع فرض شده است که میان دقت مدل برای یک نمونه و احتمال تعلق آن به b_c تناسب معکوس وجود دارد و این نتیجه، جز با پذیرش فرض اول مقاله امکان پذیر نیست.

زیربخش پنجم: ایده الگوریتم JTT

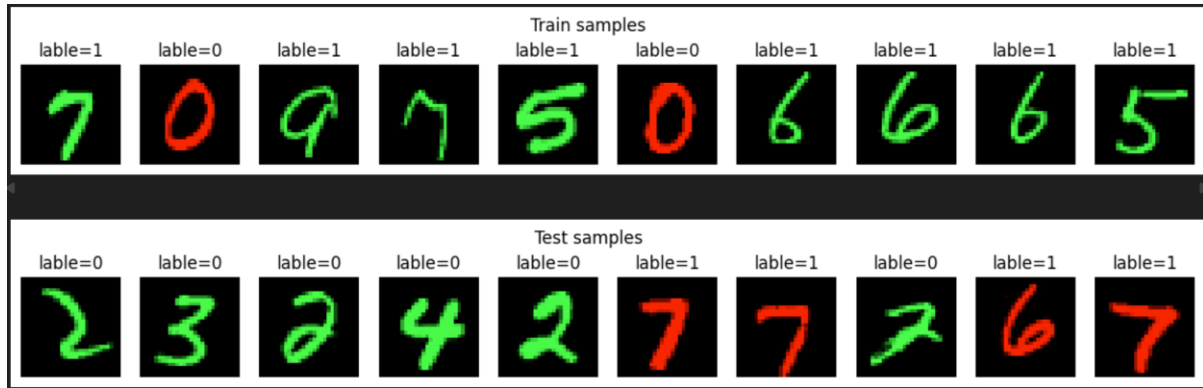
ایده مقاله JTT این است که در مرحله اول، مدل را با ERM و در چند epoch آموزش می‌دهد و سپس نمونه‌هایی که به اشتباه برچسب خورده‌اند را به عنوان نمونه‌های سخت در نظر می‌گیرد. استدلال هم این است که مدل روی نمونه‌هایی که بر خلاف همبستگی کاذب هستند، دقت کمتری دارد. سپس در مرحله دوم،

به این نمونه‌های سخت، وزن بیشتری می‌دهد و مدل را دوباره آموزش می‌دهد. بنابراین مدل مجبور می‌شود که به این نمونه‌ها توجه بیشتری کند و در نتیجه عملکردش در برابر بدترین گروه بهتر می‌شود.

بخش دوم: سوالات پیاده‌سازی

زیربخش اول: آماده‌سازی داده و مصورسازی

کد این بخش در فایل P1.ipynb در فلدر Q1 موجود است و نتیجه به صورت زیر به دست آمده است:



شکل ۵: ده نمونه رندوم از هر دیتاست پیش‌پردازش شده

زیربخش دوم: آموزش مدل پایه با ERM

کد این بخش در فایل P1.ipynb در فلدر Q1 موجود است و نتیجه به صورت زیر به دست آمده است:

```
ERM Model:  
  
ERM Train Accuracy: 97.69%  
ERM Test Accuracy: 54.07%
```

شکل ۶: دقت مدل ERM

در این بخش یک مدل MLP ساده با رویکرد ERM آموزش داده شده است. یعنی مدل فقط تلاش می‌کند که میانگین خطا روی داده‌های آموزشی را کم کند و ممکن است در گروه‌های اقلیتی، دقت خوبی نداشته باشد.

در داده‌های آموزشی، بین رنگ و کلاس یک همبستگی کاذب وجود دارد؛ یعنی اکثریت نمونه‌های کلاس قرمز و اکثریت نمونه‌های کلاس ۱ سبز هستند. به همین دلیل مدل به جای یادگیری شکل واقعی اعداد، رنگ تصویر را یاد می‌گیرد و در واقع رنگ تصویر را یک ویژگی مهم در تعیین کلاس آن قلمداد می‌کند.

حال در داده‌های تست این رابطه‌ی رنگ و کلاس کاملاً برعکس شده است. بنابراین اگر مدل به رنگ وابسته شده باشد، دقت آن در داده‌های تست کاهش می‌یابد. اختلاف زیاد بین دقت مجموعه‌های train و test (چیزی

در حدود ۴۰ درصد) نشان می‌دهد که مدل ERM، به همبستگی کاذب موجود در داده‌ی آموزشی وابسته شده است و در نتیجه در برابر تغییر این همبستگی عملکرد خوبی ندارد. پس دقت آن در مجموعه تست به شدت کاهش می‌یابد.

زیربخش سوم: استخراج مجموعه خطا (ERROR SET)

کد این بخش در فایل P1.ipynb در فلدر Q1 موجود است و نتیجه به صورت زیر به دست آمده است:

```
Train dataset size: 60000  
Error set size: 1389
```

شکل ۷: تعداد نمونه‌های error set

زیربخش چهارم: ساخت دیتاست وزن‌دار (UPSAMPLING)

کد این بخش در فایل P1.ipynb در فلدر Q1 موجود است و نتیجه به صورت زیر به دست آمده است:

```
Original train dataset size: 60000  
Error set size: 1389  
Weighted dataset size: 128061
```

شکل ۸: تعداد نمونه‌های دیتاست وزن‌دار

زیربخش پنجم: آموزش مجدد مدل

کد این بخش در فایل P1.ipynb در فلدر Q1 موجود است و نتیجه به صورت زیر به دست آمده است:

```
JTT Model:  
  
JTT Train Accuracy: 98.83%  
JTT Test Accuracy: 78.50%
```

شکل ۹: دقت مدل JTT

نتایج نشان داد که دقت مدل JTT نسبت به دقت مدل ERM حدود ۲۵ درصد افزایش یافته است. این اختلاف نشان می‌دهد که رویکرد JTT تا حد خوبی توانسته مشکل همبستگی کاذب را حل کند. در واقع در این

رویکرد با افزایش وزن نمونه‌های خطا دار، مدل را مجبور کردیم که به نمونه‌هایی که روی آنها شکست خورده است توجه بیشتری کند. این نمونه‌ها غالباً همان نمونه‌های فاقد همبستگی کاذب هستند؛ یعنی نمونه‌هایی که رنگ به تنهایی برای پیش‌بینی درست کافی نیست. بنابراین با توجه بیشتر به این نمونه‌ها، مدل وابستگی کمتری به رنگ به عنوان یک میانبر برای تعیین کلاس پیدا می‌کند و بهتر می‌تواند ویژگی‌های اصلی و واقعی اعداد را یاد بگیرد.

البته باید توجه داشت که مشکل همبستگی کاذب به طور کامل حل نشده است و هنوز دقت مدل بر روی داده‌های train و test حدود ۲۰ درصد تفاوت دارد؛ اما نسبت به مدل ERM خیلی بهتر شده است.

زیربخش ششم: پیاده‌سازی تابع زیان GCE

کد این بخش در فایل P1.ipynb در فلدر Q1 موجود است و نتیجه به صورت زیر به دست آمده است:

```
GCE error set size: 2280
Difference compared to standard ERM: 891
```

شکل ۱۰: تعداد نمونه‌های error set در رویکرد GCE

نتایج نشان داد که اندازه مجموعه Error Set افزایش یافته است. یعنی نمونه‌های بیشتری در مجموعه train بوده‌اند که مدل آنها را به درستی پیش‌بینی نکرده است.

علت این است که آموزش مدل با استفاده از تابع زیان GCE مدل را به سمت یادگیری ویژگی‌های بایاس دار و همبستگی‌های کاذب سوق می‌دهد. (علت این اتفاق در زیربخش سوم از بخش قبلی به طور کامل شرح داده شده است.) بنابراین مدل به ویژگی رنگ (به عنوان ویژگی بایاس دار) حساس‌تر می‌شود و در بین نمونه‌هایی که همبستگی رنگ و کلاس را ندارند، تعداد بیشتری را اشتباه پیش‌بینی می‌کند. پس مجموعه Error Set بزرگتر می‌شود و این می‌تواند برای شناسایی بهتر نمونه‌های چالش‌برانگیز مفید باشد و بنابراین در نهایت به مقاوم‌سازی بهتر مدل در برابر همبستگی‌های کاذب کمک کند.

سوال دوم: حملات خصمانه

بخش اول: مرور مفاهیم

سوال اول

روش‌های حمله متخاصم معمولاً به دو دسته‌ی جعبه‌سفید و جعبه‌سیاه تقسیم می‌شوند. در رویکرد جعبه‌سفید، مهاجم به اطلاعات داخلی مدل دسترسی دارد. مثلاً معماری مدل، وزن‌ها، تابع زیان و ... را می‌داند. بنابراین می‌تواند با استفاده از این اطلاعات، نویز یا تغییرات بسیار دقیقی روی ورودی ایجاد کند تا مدل را فریب دهد. اما در رویکرد جعبه‌سیاه، مهاجم به جزئیات داخلی مدل دسترسی ندارد و فقط می‌تواند ورودی به مدل بدهد و خروجی آن را مشاهده کند. در نتیجه، حمله در حالت جعبه‌سیاه سخت‌تر است، چون مهاجم باید بدون دانستن ساختار داخلی مدل، راهی برای تولید نمونه‌ی متخاصم پیدا کند.

سوال دوم

اهمیت مطالعه حملات در رویکرد جعبه‌سیاه، از آن جهت است که در دنیای واقعی هم در اکثر مواقع مهاجم به وزن‌ها و معماری داخلی مدل دسترسی ندارد و بنابراین فقط می‌تواند از تخاصم در حالت جعبه‌سیاه استفاده کند و اگر مدل در برابر این حملات مقاوم نباشد، به این معناست که حتی در صورت حفظ محرمانگی ساختارش، می‌توان آن را فریب داد. در نتیجه قابلیت اعتماد و امنیت خود را از دست می‌دهد.

سوال سوم

حملات متخاصم به دو دسته‌ی هدف‌دار و غیرهدف‌دار تقسیم می‌شوند. در حمله‌ی غیرهدف‌دار، هدف فقط این است که مدل برچسب اشتباهی را به ورودی بدهد و مهم نیست که چه برچسبی باشد؛ همین که برچسب درست نباشد کافی است. اما در حمله‌ی هدف‌دار، هدف این است که مدل، یک برچسب اشتباه از پیش تعیین شده به ورودی بدهد و نه هر برچسب اشتباهی.

بخش دوم: حمله جعبه سیاه

سوال اول

استدلال مقاله این است که چون داده‌های تصویری تعداد بعد بسیار بالایی دارند و همچنین تصاویر طبیعی به مرزهای تصمیم نزدیک هستند، نمونه‌های متخاصم تقریباً همیشه وجود دارند. بنابراین در حالت جعبه سفید، چون مهاجم به گرادیان دسترسی دارد، پیدا کردن یک تغییر کوچک برای فریب دادن مدل کار سختی نیست.

سوال دوم

دلیل اتخاذ رویکرد بردارهای متعامد این است که این بردارها، جهت‌های مستقل و غیر تکراری برای جستجو هستند. در واقع وقتی تغییرات را روی بردارهای متعامد اعمال می‌کنیم، این تغییرات مختلف همدیگر را خنثی یا بیش از حد تقویت نمی‌کنند. در نتیجه با تعداد query کمتری به هدف می‌رسیم و در عین حال اندازه perturbation بهتر کنترل می‌شود.

ایده کلی حمله نیز به این شکل است که چون جهت درست حمله را نمی‌دانیم، چند جهت مختلف را امتحان می‌کنیم و تغییر می‌دهیم و هر تغییری که احتمال کلاس درست را کم می‌کند، نگه می‌داریم. در واقع روش کلی حمله این است که در هر مرحله یک جهت تصادفی q انتخاب می‌کنیم و تصویر را کمی در آن جهت تغییر می‌دهیم. در ابتدا $x + \epsilon q$ را امتحان می‌کنیم. اگر احتمال کلاس درست کمتر شد، همین تغییر را نگه می‌داریم و اگر نشد، $x - \epsilon q$ را امتحان می‌کنیم. این روند ادامه پیدا می‌کند تا احتمال کلاس درست آن قدر کم شود که مدل تصویر را اشتباه طبقه‌بندی کند.

سوال سوم

تفاوت اصلی SimBA و SimBA-DCT در این است که جهت‌های تصادفی q از کجا انتخاب می‌شوند. در SimBA، این جهت‌ها از پایه استاندارد یا کارترین یا همان فضای پیکسل انتخاب می‌شوند؛ یعنی هر بار یک پیکسل یا یک کانال رنگی انتخاب می‌شود و مقدار آن کمی تغییر می‌کند. اما در SimBA-DCT، این جهت‌ها از پایه‌ی DCT انتخاب می‌شوند. در روش DCT، تصویر، توسط یک تبدیل کسینوسی، به الگوهای فرکانسی با فرکانس‌های مشخص تبدیل می‌شود. سپس از این فضای فرکانسی، یک الگوی کم فرکانس انتخاب می‌شود و به تصویر اضافه یا کم می‌شود.

نتایج مقاله نشان می‌دهد که SimBA-DCT در اکثر موارد با تعداد query کمتری نسبت به SimBA موفق به فریب مدل می‌شود.

سوال چهارم

تفاوت اصلی بین این دو رویکرد، در میزان اطلاعاتی است که مهاجم از مدل دریافت می‌کند. در حملات مبتنی بر برچسب، مهاجم فقط برچسب نهایی را می‌بیند و از احتمال یا میزان اطمینان مدل از درستی برچسب، اطلاعاتی دریافت نمی‌کند. به همین دلیل، این نوع از حملات سخت‌تر هستند و معمولاً به تعداد query بیشتری نیاز دارند. در مقابل، در حملات امتیازمحور، مدل علاوه برچسب، مقدار confidence یا probability کلاس‌ها را هم برمی‌گرداند. این اطلاعات برای مهاجم بسیار مفیدتر است؛ چون می‌تواند بررسی کند که آیا تغییر کوچکی که روی تصویر داده، احتمال کلاس درست را کم کرده است یا نه و بر همین اساس تصمیم بگیرد که تغییر را بپذیرد یا رد کند. بنابراین می‌توان گفت که این نوع از حملات آسان‌تر هستند و به تعداد query کمتری نیاز دارند.

سوال پنجم

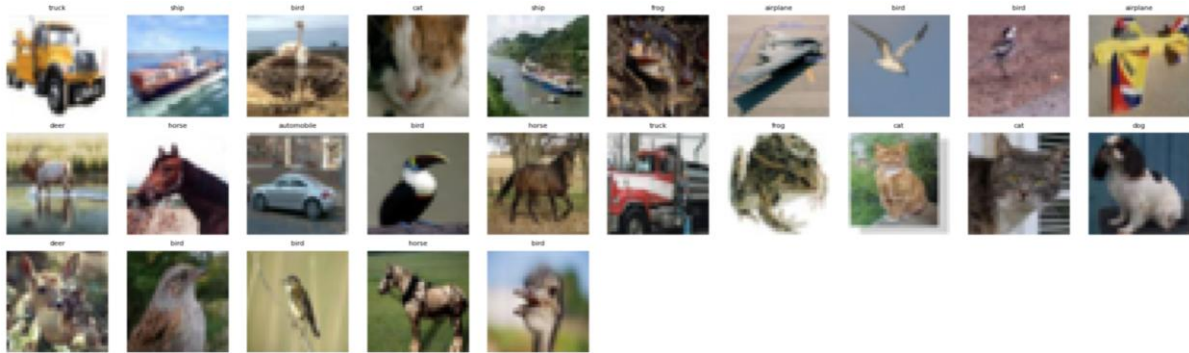
۳ معیار ارزیابی استفاده شده به شرح زیر هستند:

- ۱- Average queries: نشان‌دهنده‌ی میانگین تعداد سوالاتی است که حمله برای موفق شدن به آن نیاز دارد که هر چه کمتر باشد بهتر است چرا که هزینه پایین‌تری دارد.
- ۲- Average L₂: این معیار در واقع average perturbation L₂-Norm است که نشان‌دهنده‌ی میانگین اندازه‌ی تغییر ایجادشده روی تصویر است و هر چه مقدار آن کمتر باشد بهتر است چرا که نشان‌دهنده‌ی آن است که تصویر خصمانه، به تصویر اصلی شبیه‌تر است و تغییرات نامحسوس‌تر بوده‌اند.
- ۳- Success rate: نشان‌دهنده‌ی درصد موفقیت حمله است. یعنی حمله در چند درصد تصاویر توانسته مدل را فریب بدهد. طبیعتاً هر چه بیشتر باشد بهتر است.

بخش سوم: پیاده‌سازی حملات جعبه‌سفید

زیربخش اول: مدل و مجموعه داده

همانطور که در دستور کار آمده، تمام تصاویر مشاهده شد و دقت مدل ۱۰۰ درصد است:



شکل ۱۱: مجموعه تصاویر بارگذاری شده

```
100%|██████████| 7/7 [00:00<00:00, 14.14it/s]  
Accuracy of the model on the 25 clean images: 1.0000
```

شکل ۱۲: دقت مدل قبل از حمله

زیربخش دوم: FGSM

روش FGSM یا Fast Gradient Sign Method یک حمله‌ی جعبه‌سفید است. ایده‌ی اصلی این روش این است که تصویر را در جهتی به مقدار کمی تغییر دهیم، تا خطای مدل نسبت به برچسب واقعی بیشتر شود. وقتی خطا زیاد شود، احتمال اینکه مدل تصویر را اشتباه طبقه‌بندی کند هم بیشتر می‌شود.

فرمول ریاضی روش FGSM در حالت *untargeted* به صورت زیر است:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

شکل ۱۳: فرمول ریاضی حمله FGSM

در این فرمول، x تصویر اصلی، x_{adv} تصویر خصمانه، J تابع زیان، y برچسب واقعی، θ پارامترهای مدل و ϵ شدت حمله را مشخص می‌کند.

حال می‌توانیم مراحل روش FGSM برای ساخت نمونه *adv* را به صورت دقیق‌تر مورد بررسی قرار دهیم. در ابتدا گرادینان تابع زیان نسبت به کل تصویر ورودی محاسبه می‌شود. این گرادینان نشان می‌دهد که تغییر کوچک مقدار هر پیکسل و هر کانال رنگی، چه اثری بر *loss* مدل دارد. سپس با استفاده از تابع *sign* فقط جهت گرادینان نگه داشته می‌شود؛ یعنی برای هر مقدار تصویر مشخص می‌شود که باید کمی افزایش یابد یا کمی کاهش پیدا کند تا *loss* بیشتر شود.

اگر علامت گرادینان برای یک کانال مثبت باشد، یعنی افزایش مقدار آن کانال، باعث افزایش *loss* می‌شود؛ بنابراین به آن کانال، مقدار ϵ اضافه می‌کنیم. اگر علامت گرادینان برای یک کانال منفی باشد، یعنی کاهش مقدار آن کانال باعث افزایش *loss* می‌شود؛ بنابراین از آن کانال، مقدار ϵ کم می‌کنیم. این عملیات برای همه‌ی پیکسل‌ها و کانال‌های تصویر به صورت هم‌زمان انجام می‌شود. در نتیجه، یک نویز کوچک و جهت‌دار به تصویر اضافه می‌شود که هدف آن افزایش خطای مدل و بیشتر کردن احتمال طبقه‌بندی اشتباه است و به این ترتیب نمونه *adv* ساخته می‌شود.

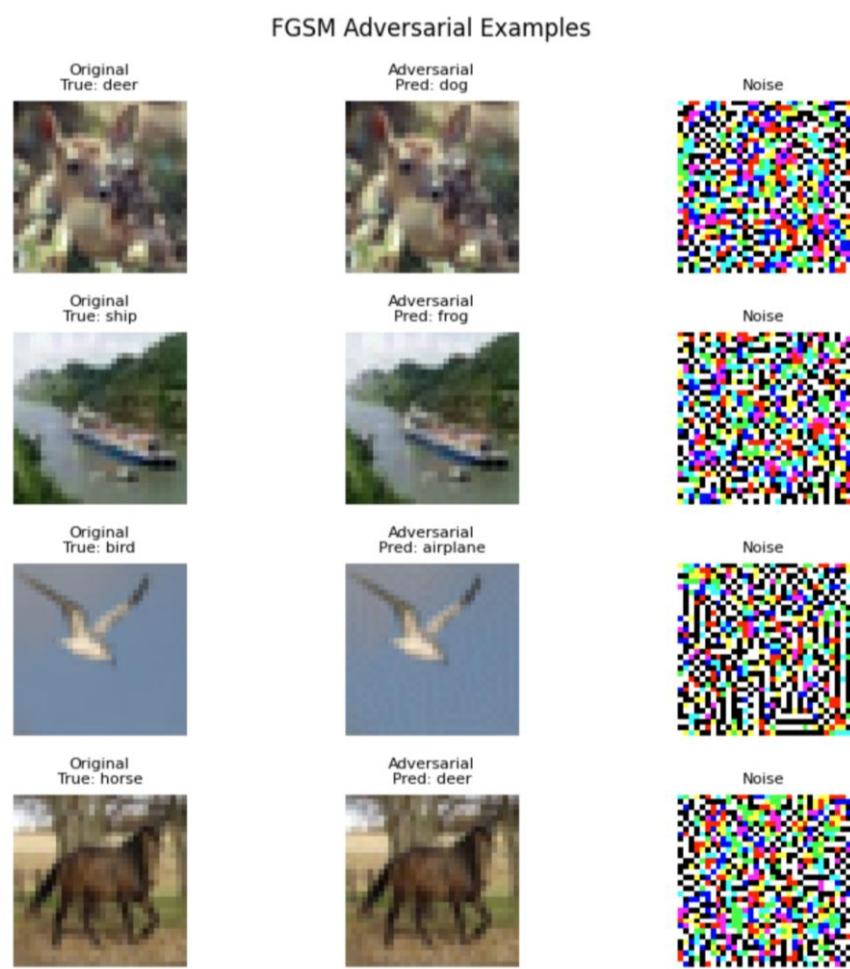
کد این بخش در فایل *P2.ipynb* در فلدر Q2 موجود است.

دقت مدل پس از حمله FGSM به صورت زیر است:

```
After FGSM Attack:  
Number of incorrect model outputs: 18  
Model Accuracy: 28.00%
```

شکل ۱۴: دقت مدل پس از حمله FGSM

۴ نمونه از تصاویر خصانه‌ای که مدل را فریب داده‌اند در تصویر زیر قابل مشاهده است:



شکل ۱۵: چهار نمونه از حملات موفق FGSM

زیربخش سوم: PGD

روش PGD یا Projected Gradient Descent یک حمله‌ی جعبه‌سفید است. ایده‌ی اصلی این روش شبیه FGSM است؛ یعنی تصویر را در جهتی تغییر می‌دهیم که خطای مدل نسبت به برچسب واقعی بیشتر شود. تفاوت اصلی این روش با FGSM این است که FGSM این تغییر را فقط در یک مرحله انجام می‌دهد، اما PGD این کار را در چندین مرحله‌ی کوچک تکرار می‌کند. به همین دلیل PGD معمولاً حمله‌ی قوی‌تری نسبت به FGSM است؛ چرا که فرصت بیشتری دارد تا تصویر را مرحله‌به‌مرحله به سمت اشتباه شدن پیش‌بینی مدل حرکت دهد.

فرمول ریاضی روش PGD در حالت untargeted به صورت زیر است:

$$x_{adv}^0 = x$$

$$x_{adv}^{t+1} = \Pi_{B_\epsilon(x)}(x_{adv}^t + \alpha \cdot \text{sign}(\nabla_x J(\theta, x_{adv}^t, y)))$$

شکل ۱۶: فرمول ریاضی PGD

در این فرمول، x تصویر اصلی، x_{adv}^t تصویر خصمانه در مرحله‌ی t ، تابع زیان، y برچسب واقعی، θ پارامترهای مدل، α اندازه‌ی هر گام و ϵ حداکثر مقدار مجاز برای نویز است. عبارت $\Pi_{B_\epsilon(x)}$ نیز به معنی projection است؛ یعنی بعد از هر گام، تصویر دوباره به محدوده‌ی مجاز اطراف تصویر اصلی برگردانده می‌شود.

حال می‌توانیم مراحل روش PGD برای ساخت نمونه‌ی adversarial را دقیق‌تر بررسی کنیم. در ابتدا تصویر خصمانه برابر با تصویر اصلی در نظر گرفته می‌شود. سپس در هر مرحله، گرادیان تابع زیان نسبت به تصویر فعلی محاسبه می‌شود. این گرادیان نشان می‌دهد که تغییر کوچک مقدار هر پیکسل یا هر کانال رنگی چه اثری بر loss مدل دارد. سپس مانند FGSM، با استفاده از تابع sign فقط جهت گرادیان نگه داشته می‌شود. اگر علامت گرادیان برای یک کانال مثبت باشد، یعنی افزایش مقدار آن کانال باعث افزایش loss می‌شود؛ بنابراین مقدار آن کانال به اندازه‌ی α افزایش داده می‌شود. اگر علامت گرادیان منفی باشد، یعنی کاهش مقدار آن کانال باعث افزایش loss می‌شود؛ بنابراین مقدار آن کانال به اندازه‌ی α کاهش داده می‌شود. این کار برای همه‌ی پیکسل‌ها و کانال‌های تصویر به صورت هم‌زمان انجام می‌شود.

اما نکته‌ی مهم در PGD این است که بعد از هر مرحله، مقدار تغییر ایجادشده نسبت به تصویر اصلی کنترل می‌شود. یعنی اگر نویز از حد مجاز ϵ بیشتر شده باشد، دوباره به محدوده‌ی مجاز برگردانده می‌شود. به این کار projection گفته می‌شود. بنابراین PGD در هر مرحله تلاش می‌کند loss را افزایش دهد، اما هم‌زمان اجازه نمی‌دهد تصویر خصمانه بیش از حد از تصویر اصلی دور شود.

در نتیجه، PGD با انجام چندین گام کوچک و کنترل شده، یک نویز جهت دار و محدود به تصویر اضافه می کند. هدف این نویز افزایش خطای مدل و بیشتر کردن احتمال طبقه بندی اشتباه است. به این ترتیب، نمونه ای adv ساخته می شود. این روش نسبت به FGSM قوی تر است، چون به جای یک حرکت سریع و یک باره، چندین بار تصویر را اصلاح می کند و هر بار آن را در محدوده ای مجاز نگه می دارد.

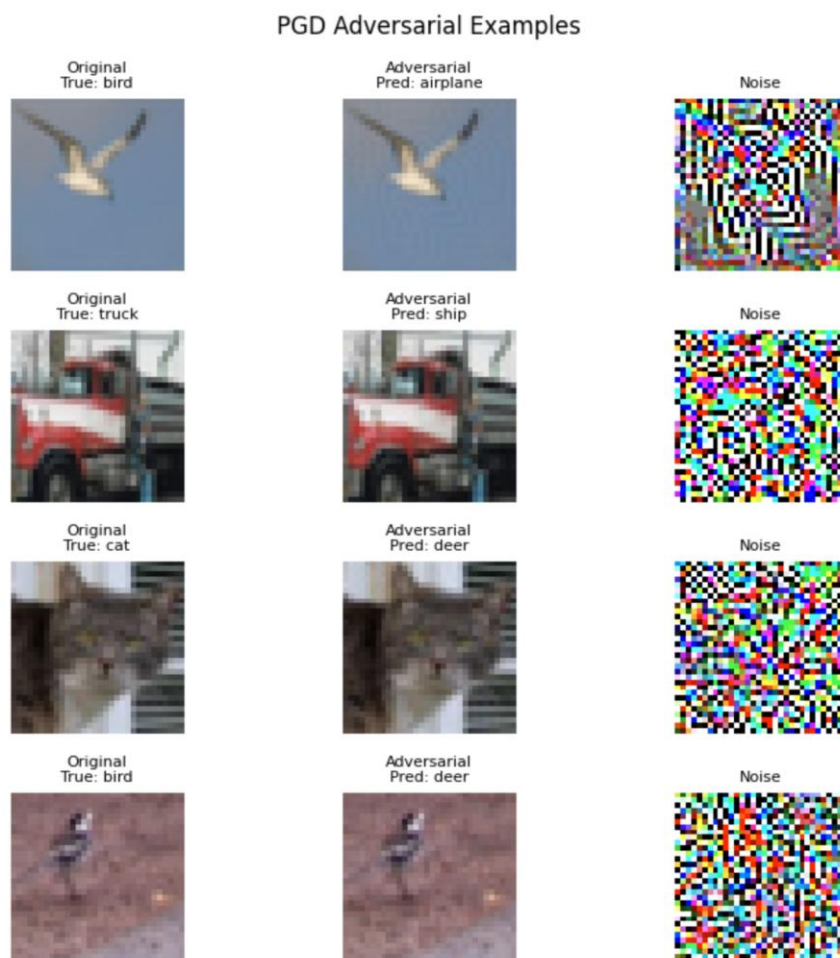
کد این بخش در فایل P2.ipynb در فلدر Q2 موجود است.

دقت مدل پس از حمله FGSM به صورت زیر است:

```
After PGD Attack:
Number of incorrect model outputs: 24
Model Accuracy: 4.00%
```

شکل ۱۷: دقت مدل پس از حمله PGD

۴ نمونه از تصاویر خصانه ای که مدل را فریب داده اند در تصویر زیر قابل مشاهده است:



شکل ۱۸: چهار نمونه از حملات موفق PGD

- [1] H. Han, S. Kim, H. Joo, S. Hong, and J. Lee. "Mitigating Spurious Correlations via Disagreement Probability" 38th Conference on Neural Information Processing Systems (NeurIPS), 2024.
- [2] E. Z. Liu, B. Haghighi, A. S. Chen, A. Raghunathan, P. W. Koh, S. Sagawa, P. Liang, and C. Finn. "Just Train Twice: Improving Group Robustness without Training Group Information" Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 2021.
- [3] C. Guo, J. R. Gardner, Y. You, A. G. Wilson, K. Q. Weinberger. "Simple Black-box Adversarial Attacks" Proceedings of the 36th International Conference on Machine Learning, Long Beach, California, PMLR 97, 2019.
- [4] OpenAI ChatGPT, used only for conceptual discussion and consultation during report preparation. No generated text was copied.
- [5] Google Gemini, used only for conceptual discussion and consultation during report preparation. No generated text was copied.
- [6] Course slide: "Robustness2".